

EXPERIMENT-12(HDOC)

C Programming

Menu-Driven Program: Even/Odd Digit Count and Sum

Step 1: Understand the Problem Statement

- Input: A positive integer, and a menu choice selecting the operation to perform.
- Output: Depending on the choice — the count of even digits and odd digits, or the sum of even digits and the sum of odd digits, computed separately.
- This is a menu-driven program using switch-case: Option 1 counts how many digits of the number are even and how many are odd; Option 2 finds the sum of all even digits and the sum of all odd digits separately; Option 3 exits the program. A digit is treated as even if it is divisible by 2 (0, 2, 4, 6, 8), and odd otherwise.

Example 1 (from problem statement): Number = 12045 → Even_dig = 3, Odd_dig = 2 (even digits: 2,0,4; odd digits: 1,5)

Example 2 (from problem statement): Number = 12345 → Sum_even = 6, Sum_odd = 9 (even digits 2+4=6; odd digits 1+3+5=9)

Rule used: A digit d is even if $d \% 2 == 0$, otherwise it is odd

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Opt 1: n=12045	Even_dig=3, Odd_dig=2	Even_dig=3, Odd_dig=2	Pass
2	Opt 2: n=12345	Sum_even=6, Sum_odd=9	Sum_even=6, Sum_odd=9	Pass
3	Opt 1: n=2468 (all even)	Even_dig=4, Odd_dig=0	Even_dig=4, Odd_dig=0	Pass

Step 3: Algorithm

1. Start the program.
2. Repeat steps 3 to 6 until the user chooses to exit.
3. Display the menu with options: (1) Count even and odd digits, (2) Sum of even digits and sum of odd digits, (3) Exit.
4. Read the user's choice.
5. If choice = 1: read a positive integer n ; repeatedly extract the last digit of a copy of n ; if the digit is even, increment evenCount, otherwise increment oddCount; remove the last digit; repeat until the copy becomes 0; display evenCount and oddCount.

6. If choice = 2: read a positive integer n; repeatedly extract the last digit of a copy of n; if the digit is even, add it to sumEven, otherwise add it to sumOdd; remove the last digit; repeat until the copy becomes 0; display sumEven and sumOdd.
7. If choice = 3: exit the loop and stop the program.
8. If choice is invalid, display an error message and show the menu again.
9. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for $n = 12045$ gives digits 1,2,0,4,5 → even digits {2,0,4} so Even_dig = 3, odd digits {1,5} so Odd_dig = 2; for $n = 12345$, even digits {2,4} sum to 6 and odd digits {1,3,5} sum to 9 — identical to the Expected Output(s) column for all 3 test cases, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* Day-13: Menu-driven program using switch-case
   Option 1: Count the number of even digits and odd digits
   Option 2: Sum of even digits and sum of odd digits, separately
*/
#include <stdio.h>

int main() {
    int choice, n, temp, digit;
    int evenCount, oddCount;
    long sumEven, sumOdd;

    do {
        printf("\n----- MENU ----- \n");
        printf("1. Count even digits and odd digits\n");
        printf("2. Sum of even digits and sum of odd digits\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a positive integer: ");
                scanf("%d", &n);

                temp = n;
                evenCount = 0;
                oddCount = 0;

                while (temp != 0) {
                    digit = temp % 10;
                    if (digit % 2 == 0)
                        evenCount++;
                    else
                        oddCount++;
                    temp /= 10;
                }

                printf("Even_dig = %d, Odd_dig = %d\n", evenCount, oddCount);
```

```

        break;

    case 2:
        printf("Enter a positive integer: ");
        scanf("%d", &n);

        temp = n;
        sumEven = 0;
        sumOdd = 0;

        while (temp != 0) {
            digit = temp % 10;
            if (digit % 2 == 0)
                sumEven += digit;
            else
                sumOdd += digit;
            temp /= 10;
        }

        printf("Sum_even = %ld, Sum_odd = %ld\n", sumEven, sumOdd);
        break;

    case 3:
        printf("Exiting program.\n");
        break;

    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 3);

return 0;
}

```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

<code>cc hdoc13.c</code>	<code>(compile)</code>
<code>./a.out</code>	<code>(execute)</code>

All 3 test cases prepared in Step 2 were executed against the compiled program by selecting the corresponding menu option. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```

Session  Actions  Edit  View  Help
(kali@kali)-[~]
$ nano hdoc13.c
(kali@kali)-[~]
$ cc hdoc13.c
(kali@kali)-[~]
$ ./a.out

—— MENU ——
1. Count even digits and odd digits
2. Sum of even digits and sum of odd digits
3. Exit
Enter your choice: 1
Enter a positive integer: 12045
Even_dig = 3, Odd_dig = 2

—— MENU ——
1. Count even digits and odd digits
2. Sum of even digits and sum of odd digits
3. Exit
Enter your choice: 2
Enter a positive integer: 12345
Sum_even = 6, Sum_odd = 9

—— MENU ——
1. Count even digits and odd digits
2. Sum of even digits and sum of odd digits
3. Exit
Enter your choice: 1
Enter a positive integer: 2468
Even_dig = 4, Odd_dig = 0

—— MENU ——
1. Count even digits and odd digits
2. Sum of even digits and sum of odd digits
3. Exit
Enter your choice: 3
Exiting program.

```

• Conclusion

In this lab session, a menu-driven C program was designed and executed to perform digit-level classification and summation on integers based on parity (even/odd):

1. **Option 1 (Even and Odd Digit Counter):** Utilized modular arithmetic ($\% 10$ and $\% 2$) inside a while loop to iteratively inspect each digit, incrementing separate counters for even digits (including zero) and odd digits.
2. **Option 2 (Even and Odd Digit Summation):** Applied a similar loop construct to extract digits and dynamically accumulate values into long integer accumulators (sumEven and sumOdd) according to their parity.

The menu-driven workflow was controlled using a do-while loop combined with a switch-case structure. The program compiled without warnings using gcc in a Linux environment, and all terminal execution results matched the values recorded in the test case table.